

Nota: O exame deve ser respondido em 3 folhas de prova separadas:

- 1 Folha para responder às três perguntas Teóricas
- 1 Folha para responder à 1ª pergunta da Prática
- 1 Folha para responder à 2ª pergunta da Prática

Duração Total Exame (T + P) : 2h:00m + 30m tolerância

19 de Junho de 2024

Parte Teórica

- 1** Um tipo mais comum de discos rígidos é o disco *Winchester*, composto por uma unidade selada com um conjunto de pratos sobrepostos, dentro de uma caixa de metal com uma pequena separação entre eles. Considere um disco deste tipo com apenas um prato de dupla face, 128 sectores, 1024 cilindros onde cada pista armazena 32KBytes.

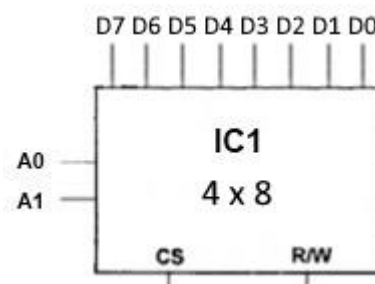
Calcule:

(2 Val)

- a) Capacidade de armazenamento de cada cilindro.
- b) Capacidade de armazenamento total do disco.

- 2** Considere o circuito integrado de memória RAM da figura, onde A₁, A₀ representam linhas de endereço, D₇, D₆, D₅, D₄, D₃, D₂, D₁, D₀ representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Select*.

Pretende-se dimensionar uma memória capaz de armazenar 13 bytes. Faça um esboço, associando múltiplos circuitos integrados do tipo IC1, de forma a obter a memória RAM pretendida. Deverá ser indicada explicitamente a linha de CS (*Chip Selection*) da memória resultante.



(2 Val)

- 3** O **Wear Leveling** é uma técnica essencial para maximizar a vida útil de dispositivos de memória **Flash**.

(3 Val)

- a) Esta técnica é necessária para contornar uma das limitações da memória **Flash**. Indique que tipo de limitação se trata.
- b) Descreva de forma simplificada o princípio de funcionamento desta técnica de **Wear Leveling**.
- c) Indique em que tipo de elementos da arquitetura do computador esta técnica é utilizada.

Parte Prática

- Desenvolva um programa em linguagem Assembly 8086 que dada uma cadeia de caracteres ASCII, **Frase1**, construa a partir desta uma nova cadeia de caracteres, **Frase2**, sem copiar para a segunda os caracteres entre parêntesis, nem os próprios parêntesis. Para a resolução do problema deve assumir que não existem parêntesis encadeados e que **Frase1** termina com o carácter '\$'. **(2,5 Val.)**

Exemplo

Frase1:

A UC (unidade curricular) de TAC (Tecnologias e Arquiteturas de Computadores) vai ter uma taxa de aprovação elevada \$

Frase2:

A UC de TAC vai ter uma taxa de aprovação elevada

- Considerando o tradicional jogo das minas, "*MineSweeper*", construa o procedimento **calcMinas** em *Assembly* que escreva, numa determinada célula, cuja posição é especificada através de linha coluna, o número de minas adjacentes existentes, caso a célula esteja vazia. Em caso de presença de mina (*) na célula, deverá ser colocado na célula **M**. Considere que o tabuleiro de jogo corresponde ao monitor do computador.

O procedimento a implementar **deve utilizar o acesso direto à memória de vídeo para realizar o output ou aceder à informação do monitor (não é permitida a utilização de interrupções)**. Como parâmetros de entrada, deve receber a posição no monitor (linha e coluna), através dos registos AL e AH, respetivamente. Como parâmetro de saída, deverá devolver no registo CL o número de minas, ou em caso de existir uma mina na posição em análise, deverá devolver *. O procedimento deve colocar na posição do ecrã (AL, AH) o número de minas adjacentes se na posição do ecrã (AL, AH) não existir qualquer valor na célula, ou escrever M se existir a mina.

Veja o exemplo de utilização do procedimento apresentado em baixo (o resultado encontra-se a negrito).

```
mov AL, 2
mov AH, 4
call calcMinas
/*Devolve 2 em CL*/

mov AL, 3
mov AH, 6
call calcMinas
/*Devolve 4 em CL*/
```

	0	1	2	3	4	5	6	7	...
0						*			
1			*	*					
2					2		*	*	
3		0			*		4	*	
4							*		
5				3	*		M		
6			*	*					
7									
:									

```
mov AL, 3
mov AH, 1
call calcMinas
/*Devolve 0 em CL*/

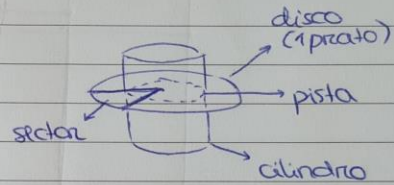
mov AL, 5
mov AH, 6
call mostraMinas
/*Devolve * em CL*/
```

(2,5 Val.)

```
.8086
.model small
.stack 2048
DATA_HERE SEGMENT
...
DATA_HERE ENDS
CODE_HERE SEGMENT
    ASSUME CS:CODE_HERE, DS:DATA_HERE
START:mov ax, DATA_HERE
    mov ds, ax
    mov bx, 0b800h
    mov es, bx
    ...
    mov ah, 4ch
    int 21h
CODE_HERE ENDS
END START
```

Parte Teórica

1.
 - * 1 prato de dupla face
 - * 128 sectores
 - * 1024 cilindros
 - * cada pista armazena 32 KB

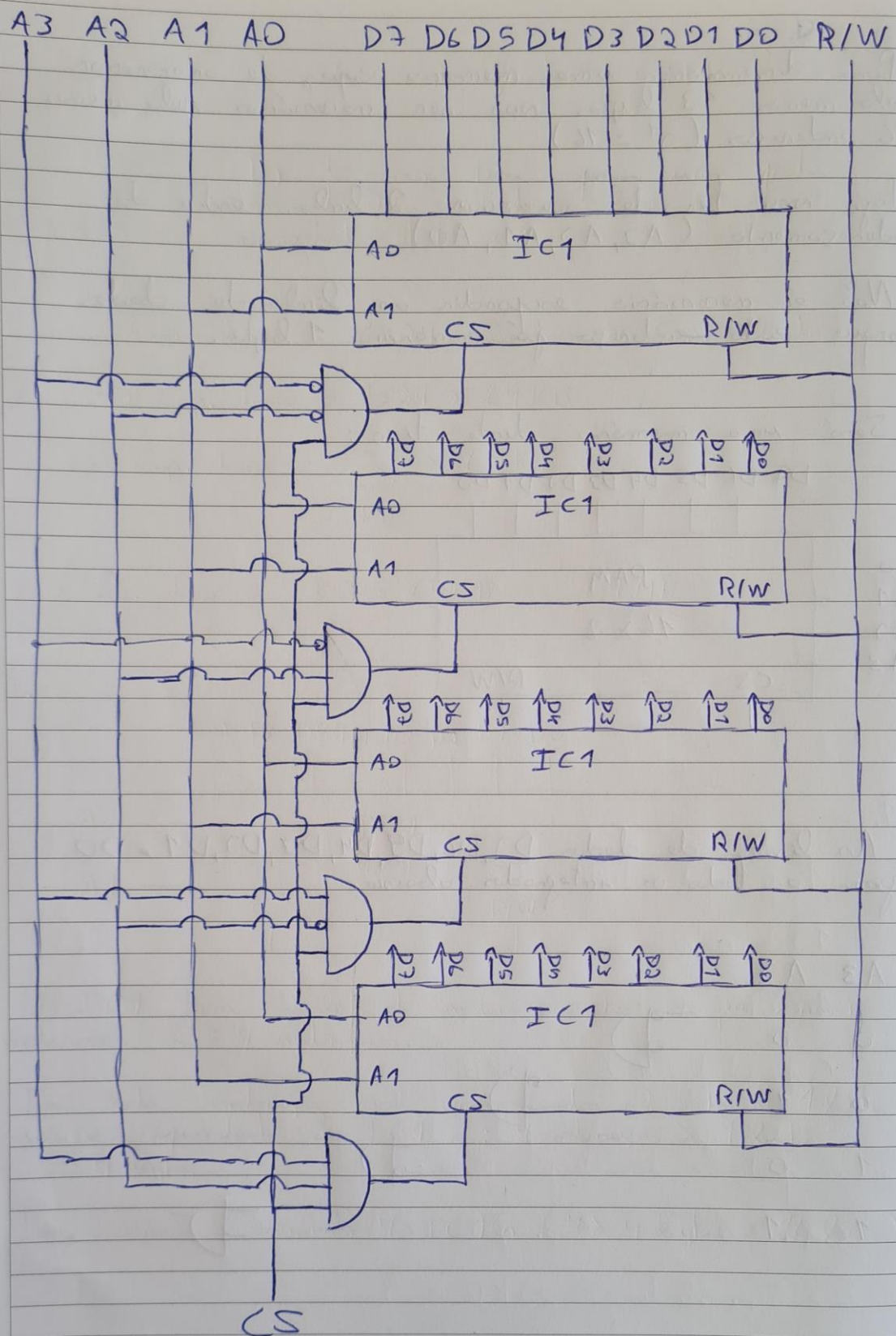


a) Capacidade de armazenamento de cada cilindro?

Cada cilindro possui 2 pistas (uma da face superior e outra da face inferior), logo a sua capacidade é de 64 KB ($2 \times 32 \text{ KB} = 64 \text{ KB}$).

b) Capacidade de armazenamento total do disco?

O disco tem 1024 cilindros, cada um com 64 KB, logo a capacidade total é de 64 KB $\times 2^{10} = 64 \text{ MB}$.



3.

a) A técnica de Wear leveling é essencial para os memórias flash porque prolonga a vida útil dos dispositivos que utilizam células de memória flash. Cada célula ~~por~~ ^{em} ~~opera~~ ^{operação} pode ser apagada e escrita um número limitado de ~~vezes~~ ^{vezes} até falhar ou ser desportada o suficiente ~~de~~ ^{para} parar de funcionar. Sempre que operações de escrita ou gravação não efetuadas na célula ela desportada - se um pouco.

b) O Wear leveling funciona ao distribuir o desgaste provocado pelas operações nas células por todas as células ~~de~~ ^{da} de memória flash do dispositivo. O controlador de memória do dispositivo rastreia o número de vezes que cada célula foi apagada ou escrita e distribui estas operações por todas as suas células de memória evitando que uma única célula seja utilizada ~~repetidamente~~ repetitivamente e evitando que seja desportada.

c) Esta técnica de Wear leveling é utilizada em qualquer dispositivo de armazenamento não volátil que utilize nas suas células a tecnologia de memória flash. Alguns destes dispositivos são: discos de unidade sólida (SSD's), pen drives USB e cartões de memória.

P1

```
.model small  
.8086  
.stack 2048
```

```
dseg segment para public 'data'  
    String1 db 'Exame de T (Tac) a testar $'  
    String2 db 50 dup (?)
```

```
dseg ends
```

```
cseg segment para public 'code'  
    assume cs:cseg, ds:dseg
```

```
Main proc  
    mov ax, dseg  
    mov ds, ax
```

```
    xor si,si  
    xor di,di  
copia: mov al, string1[si]  
    cmp al,'$'  
    je fim  
    cmp al,'('  
    je naocop  
    mov String2[di],al  
    inc di  
    inc si  
    jmp copia
```

```
naocop : inc si  
    mov al, String1[si]  
    cmp al, ')'  
    jne naocop  
    inc si  
    jmp copia
```

```
fim:  mov ah,4Ch  
    int 21h
```

```
Main endp  
cseg ends  
end Main
```

